



اُنِيُوْ سَيِّتِيْ تِيْكَوْ لُوْ كِيْ مَارَا
UNIVERSITI
TEKNOLOGI
MARA

FACULTY

COLLEGE OF COMPUTING, INFORMATICS AND MATHEMATICS

PROGRAMME

CDCS255 - BACHELOR OF COMPUTER SCIENCE (HONS.) COMPUTER NETWORKS

CODE/SUBJECT/CLASS

ITT440

NETWORK PROGRAMMING

(NBCS2555A)

PREPARED FOR:

Performance Testing and Bottleneck Analysis of DummyJSON Products

API Using Apache JMeter

LECTURER'S NAME:

ENCIK SHAHADAN BIN SAAD

PREPARED BY:

Name	Matrix No.
Muhamad Shahmi Bin Moktahr	2024727655

TABLE OF CONTENTS

No.	Topic	Pages
1.	INTRODUCTION	1
2.	OBJECTIVE	
3.	TOOL SELECTION	
4.	TARGET API DESCRIPTION	2
5.	TEST ENVIRONMENT	
6.	METHODOLOGY	
7.	TEST SCENARIOS	3
8.	KEY PERFORMANCE INDICATORS (KPIs)	5
9.	RESULT	6
10.	ANALYSIS	8
11.	BOTTLENECK ANALYSIS	9
12.	RECOMMENDATIONS	10
13.	CONCLUSION	

INTRODUCTION

Performance testing is an important process used to evaluate the responsiveness, scalability, and stability of systems under different workloads. Modern web applications commonly use REST APIs to exchange data, making API performance a critical factor in ensuring system reliability and user satisfaction.

This project evaluates the performance of the DummyJSON Products API using Apache JMeter under different traffic conditions. Three performance testing techniques were conducted, namely Load Testing, Stress Testing, and Spike Testing.

The project focuses on analysing response time, throughput, and error rate to identify system behaviour and potential bottlenecks under concurrent user traffic.

OBJECTIVES

The objectives of this project are:

- To evaluate the performance of DummyJSON Products API under different workloads
- To analyse response time and throughput
- To identify bottlenecks during high traffic conditions
- To observe API behaviour during sudden traffic spikes
- To demonstrate the use of Apache JMeter for REST API performance testing

TOOL SELECTION

Apache JMeter was selected due to the following advantages:

- Open-source performance testing tool
- Supports HTTP and REST API testing
- Provides graphical performance reports
- Simulates concurrent virtual users
- Suitable for scalability and stress testing

TARGET API DESCRIPTION

The selected API is the DummyJSON Products API.

API Endpoint: <https://dummyjson.com/products>

The API provides product-related information in JSON format, including product title, category, price, stock, and rating information. The API is publicly accessible and suitable for educational performance testing purposes.

TEST ENVIRONMENT

Component	Specification
Operating System	Windows 11
Processor	Intel Core i5
RAM	8 GB
Tool	Apache JMeter 5.6
API	DummyJSON Products API

METHODOLOGY

The testing methodology involved the following steps:

1. Creating Thread Groups in Apache JMeter
2. Configuring HTTP Request samplers
3. Executing Load Test, Stress Test, and Spike Test
4. Collecting performance metrics
5. Analyzing API behavior under concurrent traffic

The HTTP Request configuration used in this project is shown below:

Field	Value
Protocol	https
Server	dummyjson.com
Method	GET
Path	/products

HTTP Request

Name: Load Test

Comments:

Basic Advanced

Web Server

Protocol [http]: https Server Name or IP: dummyjson.com Port Number:

HTTP Request

GET Path: /products Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Configuration as shown in Figure 1.

TEST SCENARIOS

1. LOAD TEST

Purpose:

Evaluate API performance under normal traffic conditions.

Setting	Value
Users	50
Ramp-up	30 seconds
Duration	5 minutes

Thread Group

Name: Load Test

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 50

Ramp-up period (seconds): 30

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds): 300

Startup delay (seconds):

2. STRESS TEST

Purpose:

Determine API behaviour under excessive workloads.

Setting	Value
Users	150
Ramp-up	60 seconds
Duration	5 minutes

Thread Group

Name:

Stress Test

Comments:

Action to be taken after a Sampler error

☒ Continue

☐ Start Next Thread Loop

☐ Stop Thread

☐ Stop Test

☐ Stop Test Now

Thread Properties

Number of Threads (users):

150

Ramp-up period (seconds):

60

Loop Count:

☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds):

300

Startup delay (seconds):

3. SPIKE TEST

Purpose:

Analyse API performance during sudden traffic increases.

Setting	Value
Users	150
Ramp-up	5 seconds
Duration	5 minutes

Thread Group

Name: Spike Test

Comments:

Action to be taken after a Sampler error
☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 150

Ramp-up period (seconds): 5

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds): 180

Startup delay (seconds):

KEY PERFORMANCE INDICATORS (KPIs)

The following KPIs were analysed:

- Average Response Time
- Throughput
- Error Rate
- Maximum Response Time
- 90th Percentile Response Time
- 95th Percentile Response Time

RESULT

1. Aggregate Report Results

Test Type	Avg Response Time (ms)	Max Response Time (ms)	Error Rate	Throughput
Load Test	197	596	0.00%	240.1 req/sec
Stress Test	542	2135	0.00%	248.4 req/sec
Spike Test	568	1029	0.00%	259.9 req/sec

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: results/load_test_result.csv Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/s...	Sent KB/sec
Load Test	72076	197	173	311	354	596	11	3112	0.00%	240.1/sec	10595.34	36.11
TOTAL	72076	197	173	311	354	596	11	3112	0.00%	240.1/sec	10595.34	36.11

Figure 2, shows Aggregate report for Load Test

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: results/stress_test_result.csv Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/s...	Sent KB/sec
Stress Test	74744	542	459	938	1380	2135	11	16186	0.00%	248.4/sec	10962.71	37.36
TOTAL	74744	542	459	938	1380	2135	11	16186	0.00%	248.4/sec	10962.71	37.36

Figure 3, shows Aggregate report for Stress Test

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

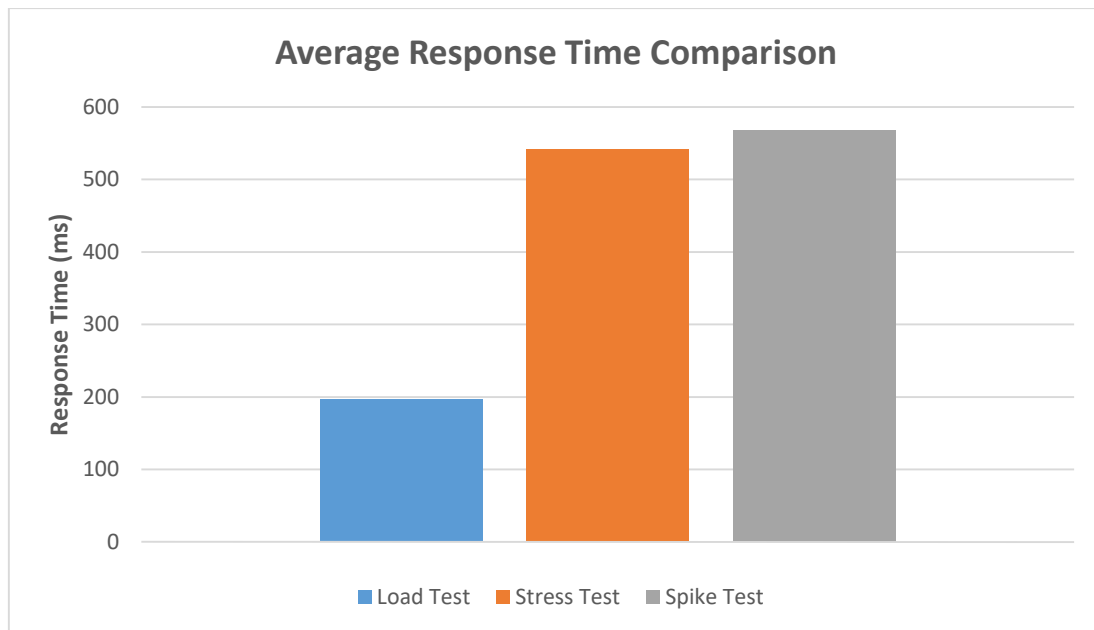
Filename: results/spike_test_result.csv Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/s...	Sent KB/sec
Spike Test	46915	568	573	597	609	1029	13	2343	0.00%	259.9/sec	11467.50	39.08
TOTAL	46915	568	573	597	609	1029	13	2343	0.00%	259.9/sec	11467.50	39.08

Figure 4, shows Aggregate report for Spike Test

2. Response Time Comparison

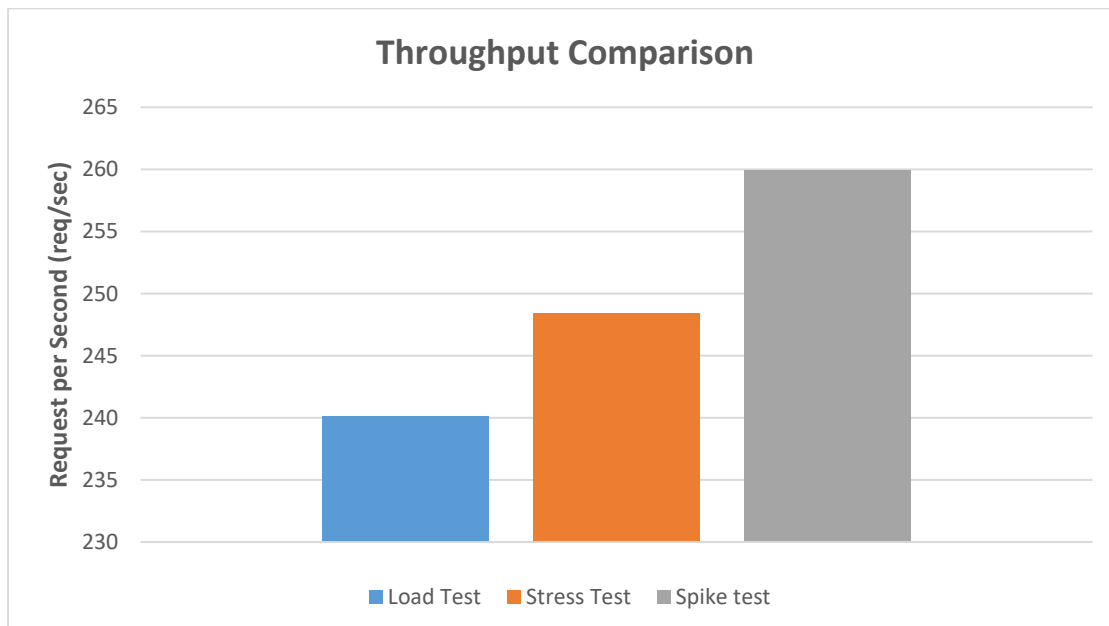
Comparison of average response times across different testing scenarios.



The graph shows that response time increased significantly during Stress Test and Spike Test due to higher concurrent traffic.

3. Throughput Comparison

Comparison of throughput across Load, Stress, and Spike testing scenarios.

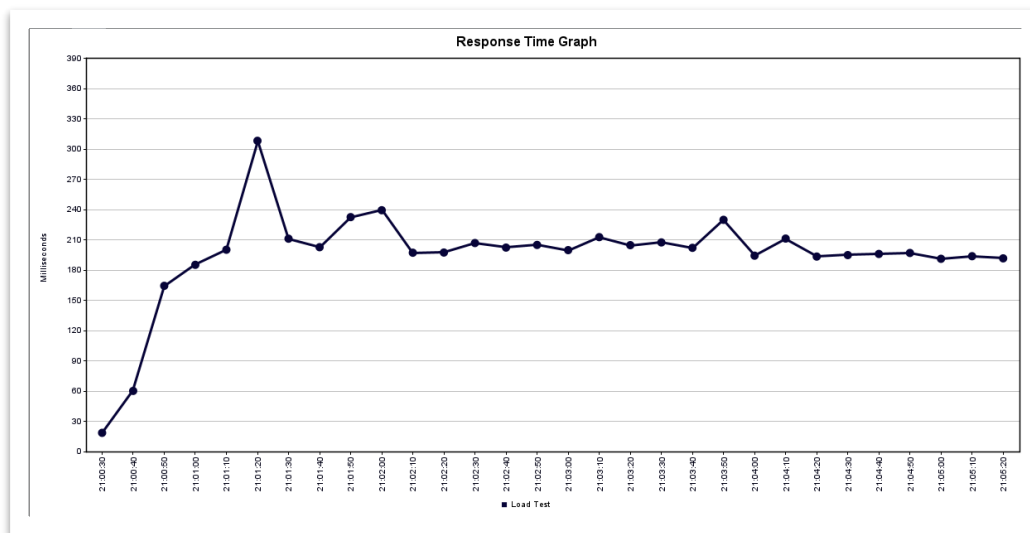


The throughput graph shows that the API handled more requests per second during higher traffic conditions.

ANALYSIS

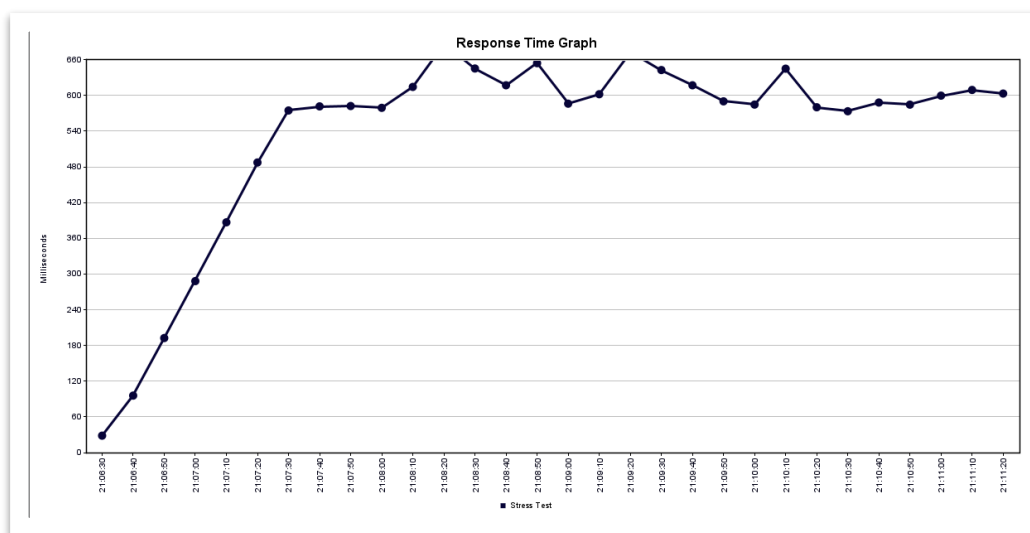
1. Load Test Analysis

The Load Test achieved an average response time of 197 milliseconds while maintaining a throughput of approximately 240 requests per second. No request failures were recorded, indicating that the API handled normal traffic efficiently.



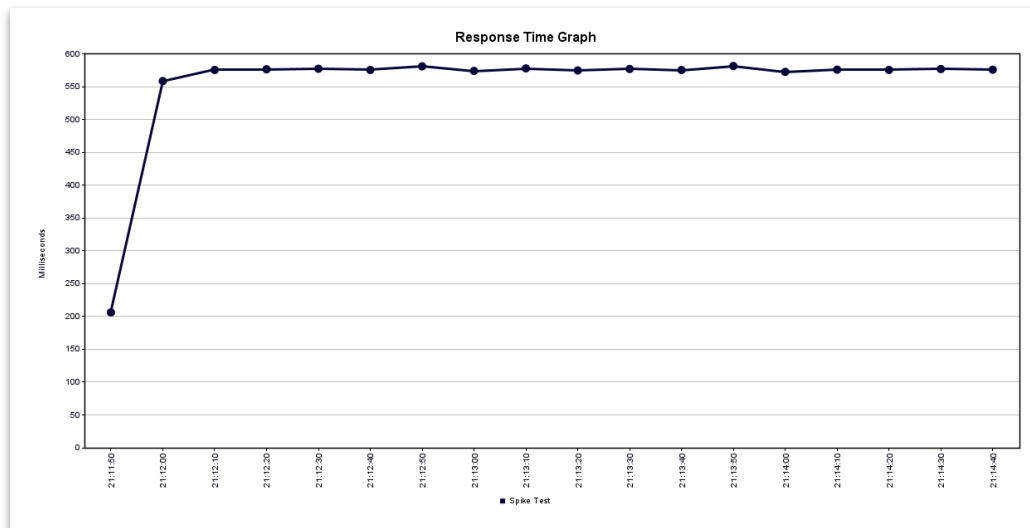
Stress Test Analysis

During the Stress Test, the average response time increased to 542 milliseconds. This increase was expected due to the larger number of concurrent users. Despite the heavier workload, the API maintained a 0.00% error rate, demonstrating strong stability and scalability.



Spike Test Analysis

The Spike Test simulated a sudden increase in traffic. The average response time increased slightly to 568 milliseconds. Although response time increased compared to the Load Test, the API successfully processed all requests without any failures.



Overall Analysis

The results show that response time increased as workload intensity increased. However, the API maintained a perfect reliability record with a 0.00% error rate across all testing scenarios. This indicates that the DummyJSON Products API is highly stable and capable of handling concurrent traffic effectively.

BOTTLENECK ANALYSIS

Although no request failures occurred, several performance limitations were observed:

- Increased response time under higher workloads.
- Longer processing delays during stress conditions.
- Higher maximum response times during peak traffic.
- Potential network latency effects.

These observations suggest that workload increases primarily affect response time rather than reliability.

RECOMMENDATIONS

The following recommendations are proposed:

1. Implement caching mechanisms.
2. Use Content Delivery Networks (CDN).
3. Optimize API response payloads.
4. Apply load balancing strategies.
5. Continuously monitor API performance.

CONCLUSION

This project successfully evaluated the performance and scalability of the DummyJSON Products API using Apache JMeter.

The API maintained a 0.00% error rate throughout all testing scenarios, demonstrating excellent reliability. Although response times increased during Stress and Spike testing, the API remained stable and responsive under concurrent traffic conditions.

The results support the hypothesis that the API can handle increased workloads successfully while maintaining service availability and reliability.